

AAKAAR — Low-Level Design (v1)

Module-by-module reference for the planner, the executor, the capability catalog, and the signal hub. Code identifiers, file paths, and wire formats are all English; this document follows the same convention.

1. Codebase layout

```
Aakaar/  
  aakar/                                # backend service (FastAPI)  
    aakar/  
      api/                              # routers, schemas, deps, app factory  
        routers/                        # auth, admin, chat, chat_sessions, runs, vault, objects  
        repositories/                  # SQLAlchemy session-scoped data access  
        schemas.py                     # Pydantic request and response models  
        app.py                         # FastAPI app factory  
        deps.py                        # auth + db dependencies  
      planner/  
        agentic/                       # multi-step agent loop  
        openai_impl.py                 # one-shot planner using strict JSON  
        prompt.py                      # prompt templates and registry rendering  
        llm.py                         # OpenAI client wrapper  
        capability_index.py            # FAISS-backed semantic index  
      interpreter/  
        executor.py                    # protocol + LocalExecutor implementation  
        activities/                    # primitive activities the executor dispatches  
      capabilities/  
        web_login/  
        file_download/  
        file_upload/  
        registry.py                    # capability + action + control catalog  
      workers/  
        browser/                       # playwright-backed browser worker  
        http/                          # httpx-backed http worker  
      models/                          # SQLAlchemy ORM models  
      migrations/                      # alembic  
      tests/  
  aakar-web/                           # tenant frontend (Vite + React + TS)  
  admin-app/                           # third-party bank-ops mock (demo fixture, not the super
```

The backend is a single deployable. The planner, executor, capabilities, and workers all live in-process behind clean module boundaries.

2. API router map

Router	Mount point	Notes
<code>auth</code>	<code>/auth</code>	login, refresh, logout, /me
<code>admin</code>	<code>/admin</code>	tenant-scoped admin (user CRUD, grants)
<code>chat</code>	<code>/chat</code>	turn-based chat that drives the planner
<code>chat_sessions</code>	<code>/chat/sessions</code>	persisted chat threads
<code>runs</code>	<code>/runs</code>	run get, cancel, events SSE
<code>workflows</code>	<code>/workflows</code>	saved workflow CRUD + version listing + start a run
<code>vault</code>	<code>/vault</code>	per-tenant credential entries
<code>objects</code>	<code>/objects</code>	artifact upload / download
<code>superuser</code>	<code>/superuser</code>	cross-tenant admin surfaces

All routers depend on `deps.get_db` and `deps.require_user`. Tenant scoping is enforced inside repositories, not by the router, so a misuse at the router layer cannot leak rows from another tenant.

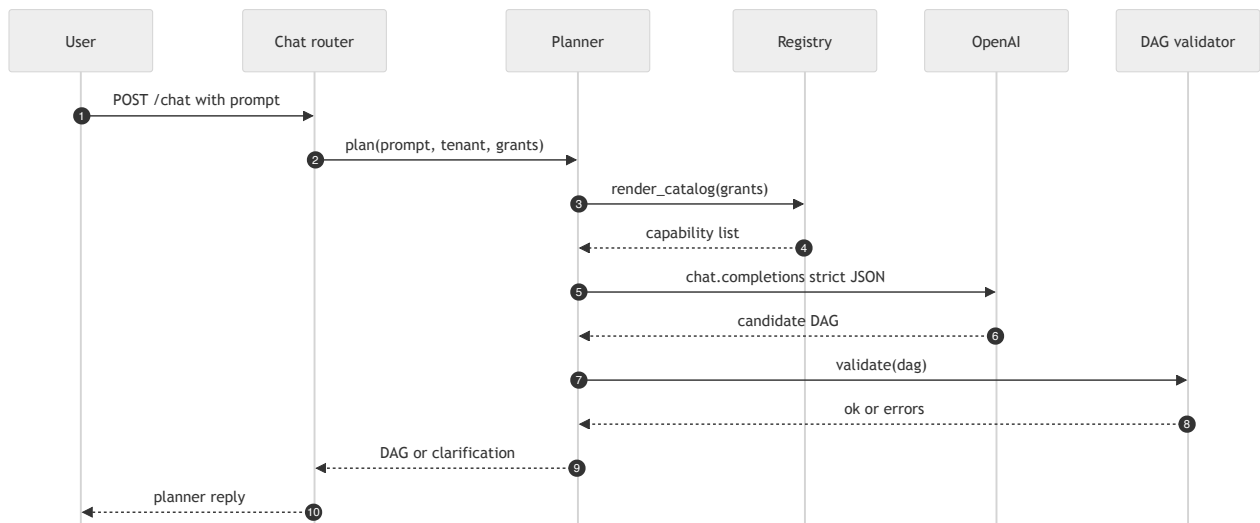
3. Planner

The planner has two strategies, picked per turn:

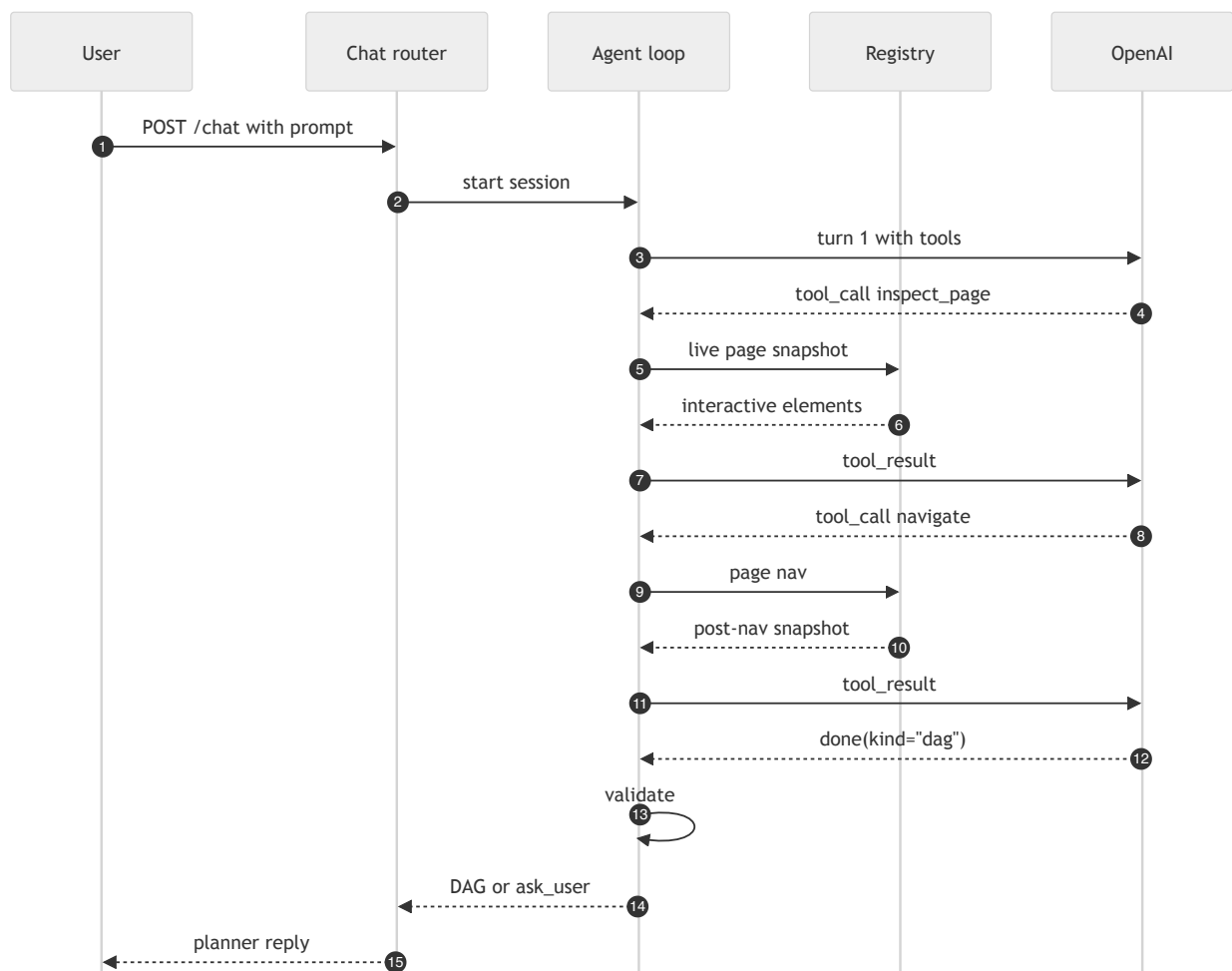
- **One-shot.** The whole prompt is rendered into a single chat completion with strict JSON mode. The model returns a complete DAG. This path is fast and used for prompts that fit one of the well-known templates.
- **Agentic.** The model is allowed to issue tool calls in a loop: `inspect_page`, `navigate`, `login_with_grant`, `done`. The loop ends when the model emits a final DAG or asks a clarifying question.

Both strategies share the registry rendering in `prompt.py` so they always see the same allowed verbs.

3.1 One-shot planner sequence

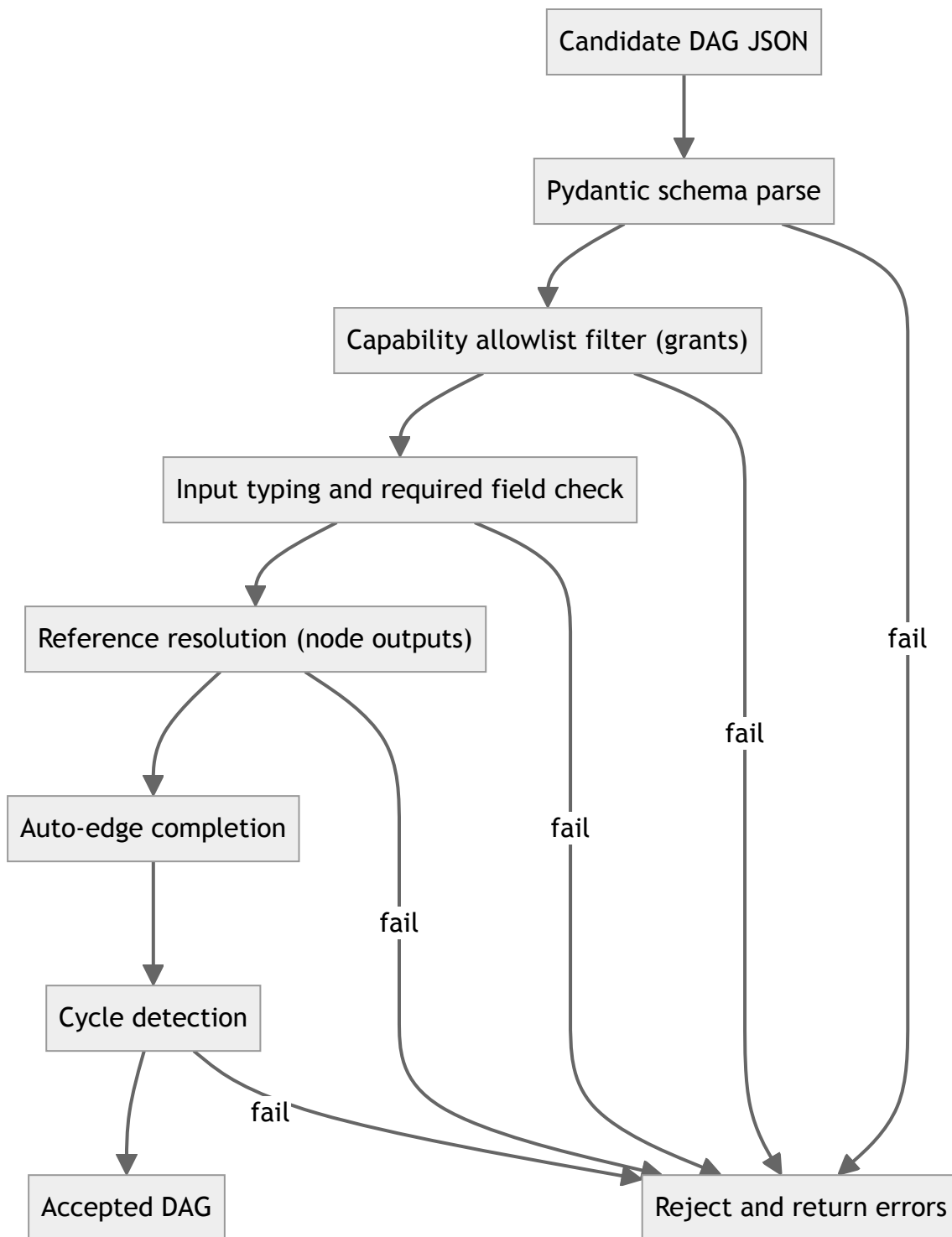


3.2 Agentic planner sequence



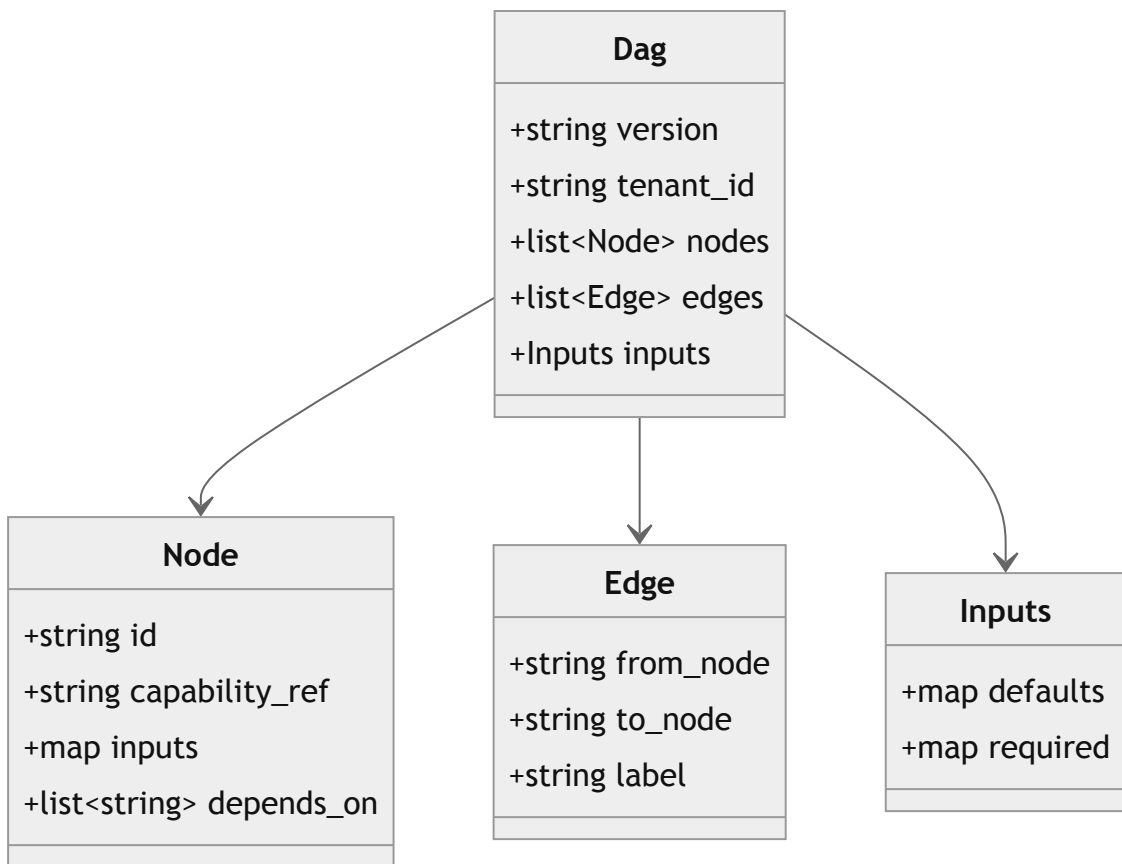
3.3 Validation pipeline

The planner never trusts the model. Every candidate DAG passes through a fixed pipeline:



Auto-edge completion is the one piece of leniency: if a node references `${node_a.output}` but the model forgot to declare an edge from `node_a`, the validator inserts the edge.

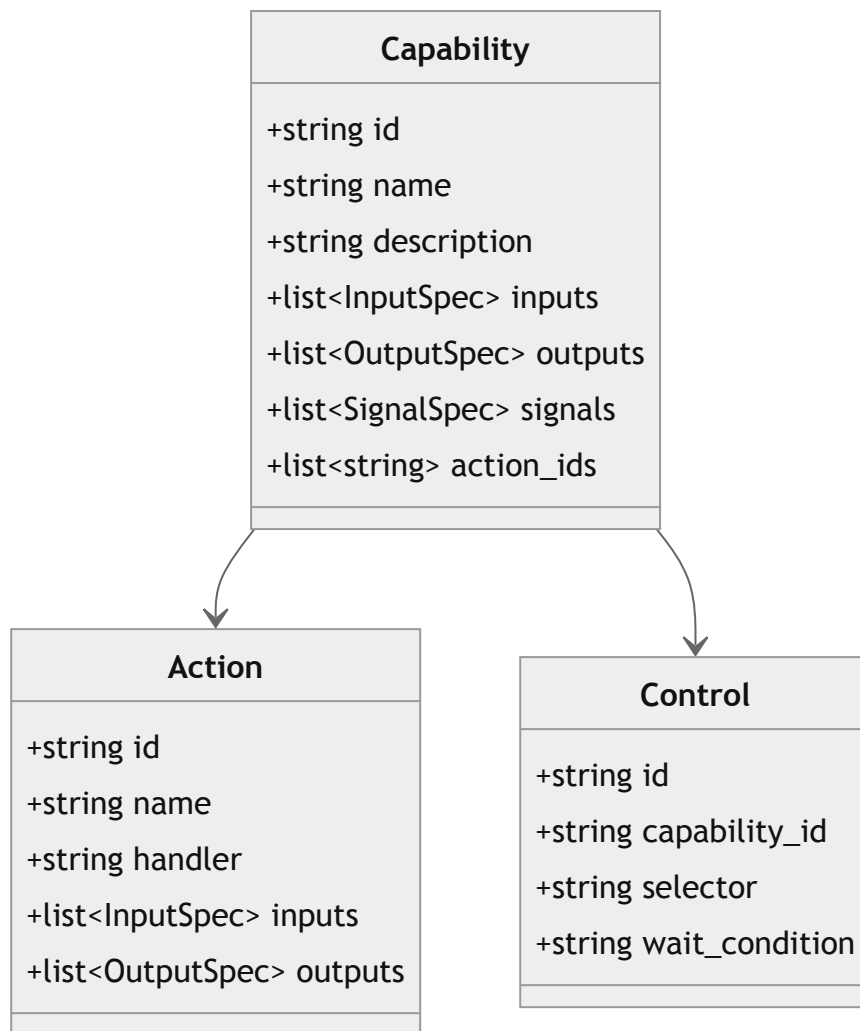
4. DAG types



`Node.inputs` accepts both literal values and reference expressions of the form `${node_id.output_field}`. References are resolved at run-time by the executor.

5. Capability registry

Capabilities, actions, and controls live in three tables.

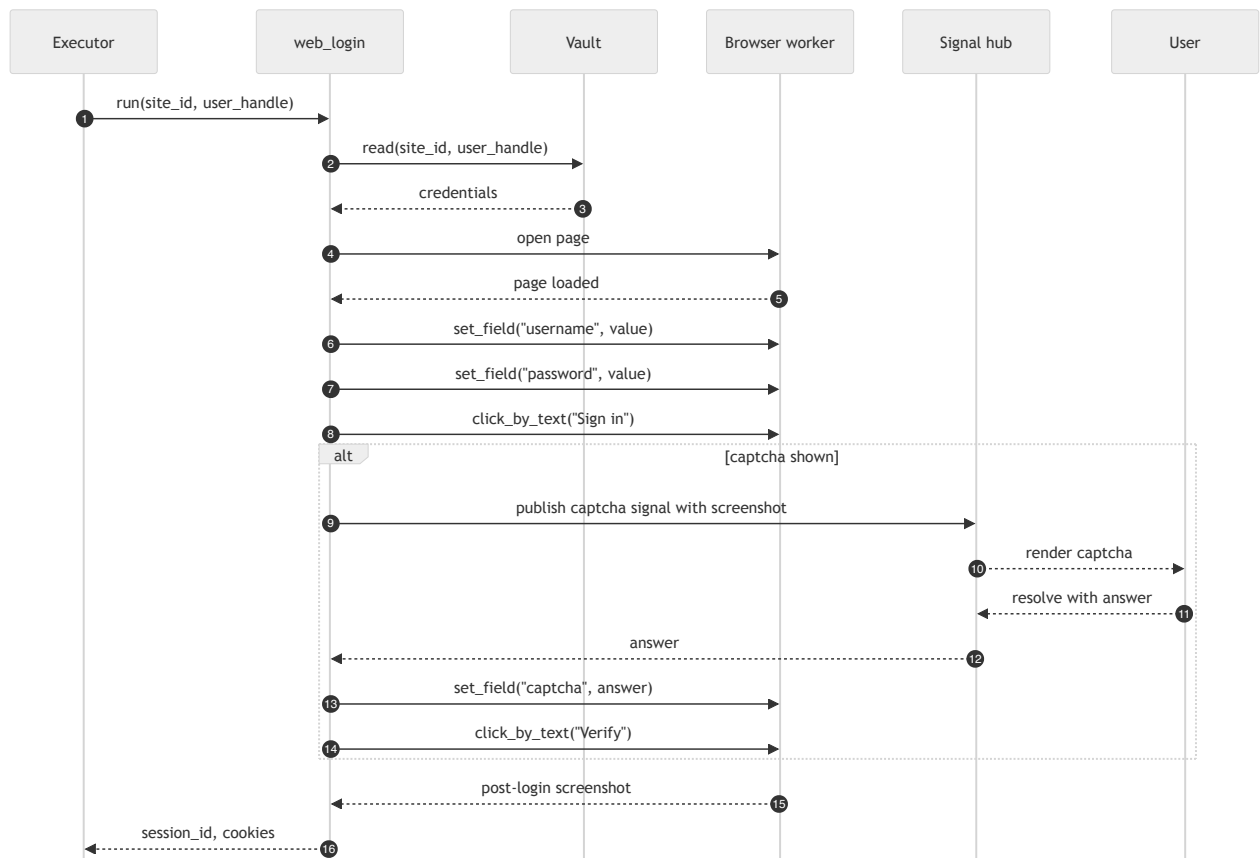


Each capability has a Python module under `aakar/capabilities/<id>/` that exports:

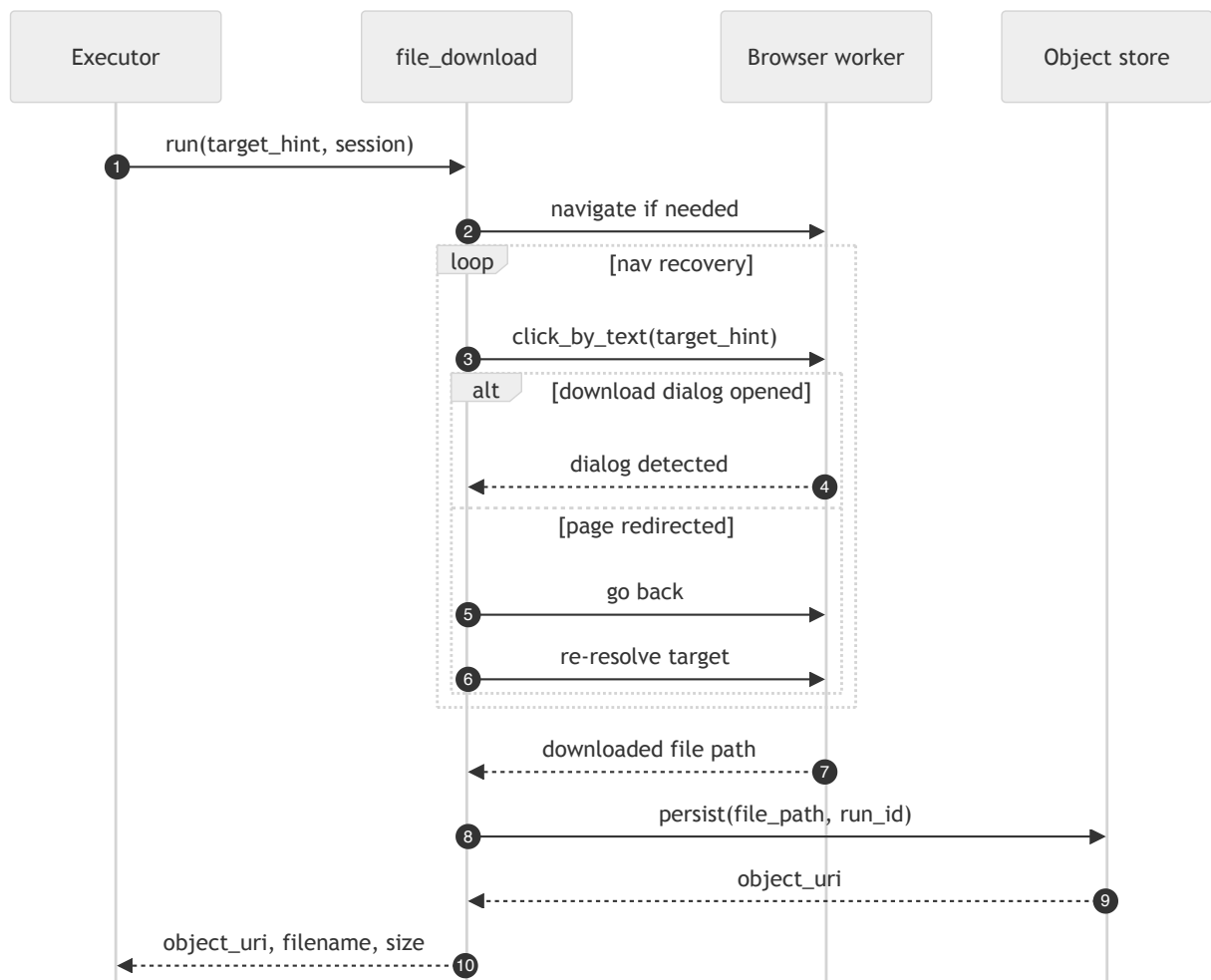
- `INPUTS` — Pydantic input schema.
- `OUTPUTS` — Pydantic output schema.
- `SIGNALS` — list of signal specs published.
- `run(inputs, ctx) -> outputs` — async entrypoint.

The registry is loaded at process start. The planner sees only what is registered for the tenant's grants.

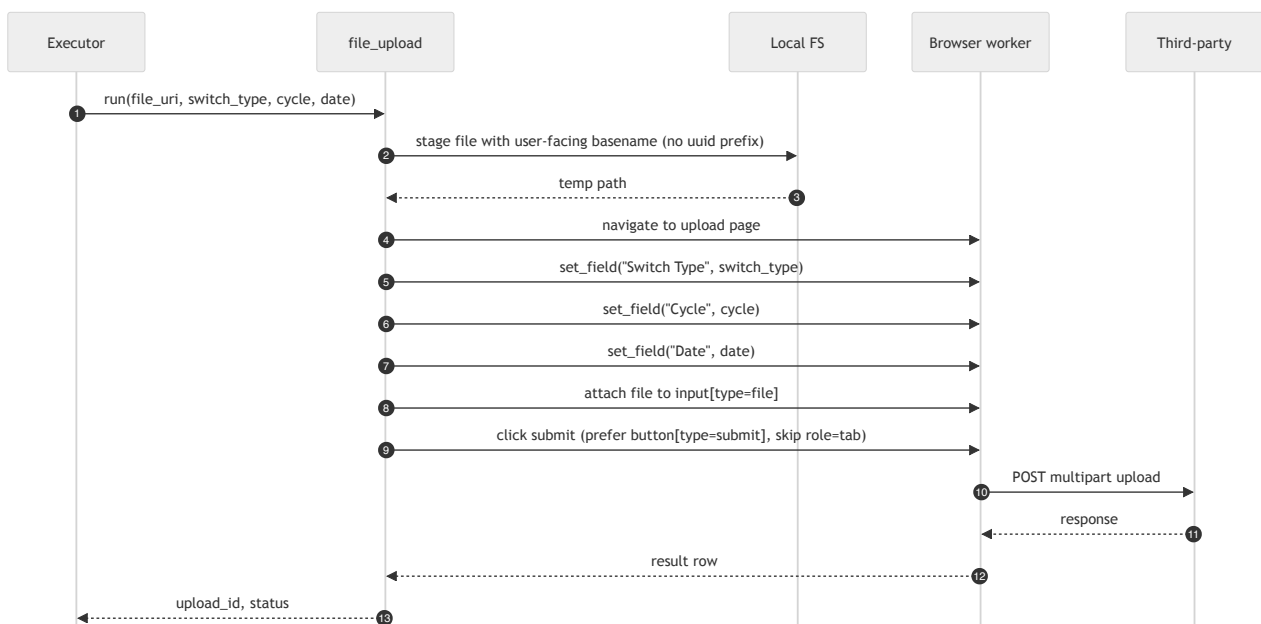
6. Capability deep dive: `cap.web_login`



7. Capability deep dive: `cap.file_download`



8. Capability deep dive: `cap.file_upload`

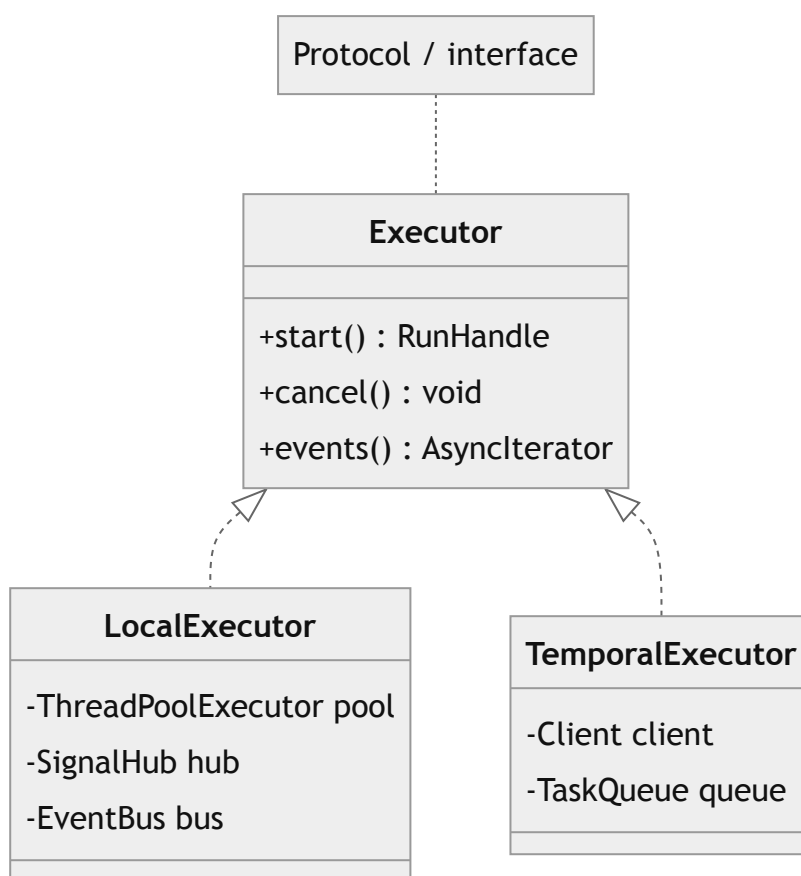


Three specific lessons baked into v1:

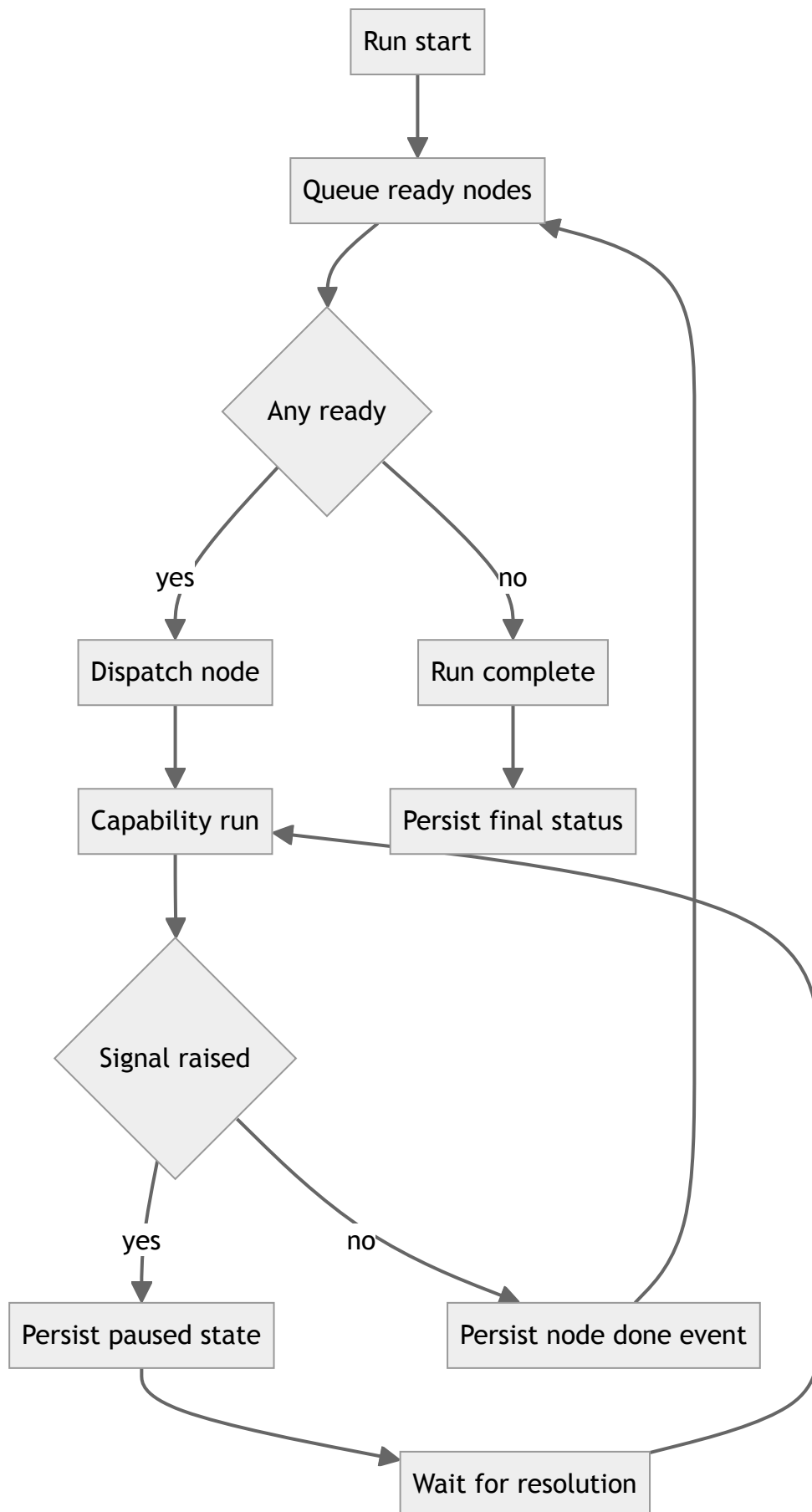
- The local temp file preserves the original suffix. Some upload endpoints reject `' '` as an extension and return 415.
- The staged copy uses the **user-facing basename** (recovered from the storage key's `<uuid32hex>_<name>.ext` pattern), so the third party records `biller_transactions_2026_05.csv` rather than `tmpXXXXXX.csv`.
- `click_by_text("Upload")` on a recon-upload page could match both the "Upload" tab and the "Upload" submit button. The capability resolves the click using a custom JS resolver that prefers `button[type='submit']` and skips elements with `role="tab"`.

9. Executor

The Executor Protocol is small and stable:



v1 ships `LocalExecutor`. It walks the DAG topologically, dispatches each ready node to its capability, persists `RunEvent` rows, and pushes them to subscribers (the SSE endpoint and the audit log). `TemporalExecutor` is a Phase 2 roadmap item — comment only today, no class.



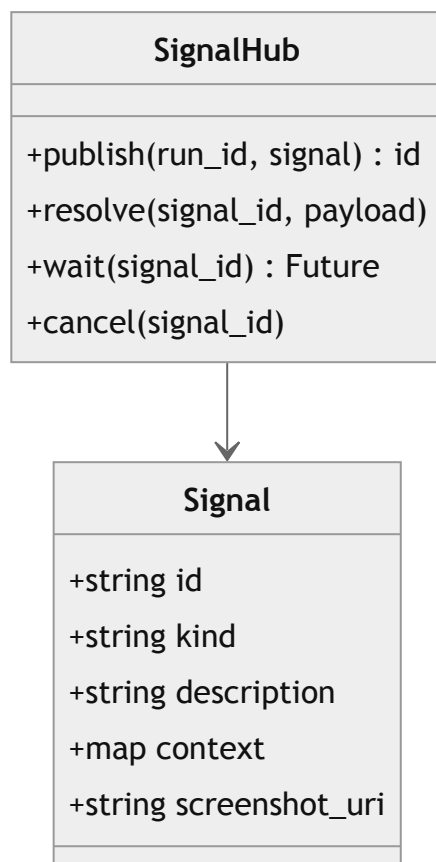
9.1 Grant-defaults merge (runtime input shaping)

Before dispatching a `cap.*` node, the executor merges the grant's `input_defaults` into the node's `inputs`. The merge treats `None` and `""` as "missing" so a planner-emitted `login_url: null` does not override a real default from the vault. Real non-empty values from the DAG always win.

9.2 Stateful chain ordering

Browser sessions are mutable: navigating, clicking, filling, and uploading each change the visible page. Nodes that operate on the same session must form a strict linear chain (not a parallel fan-out under a single login). The planner prompt enforces this; the executor honors whatever edge structure the validated DAG presents.

10. Signal hub (HITL)



Signal kinds in v1: `captcha`, `picker`, `otp`, `confirm`. Each resolution is persisted as a `RunEvent` so the audit log preserves who resolved what and when.

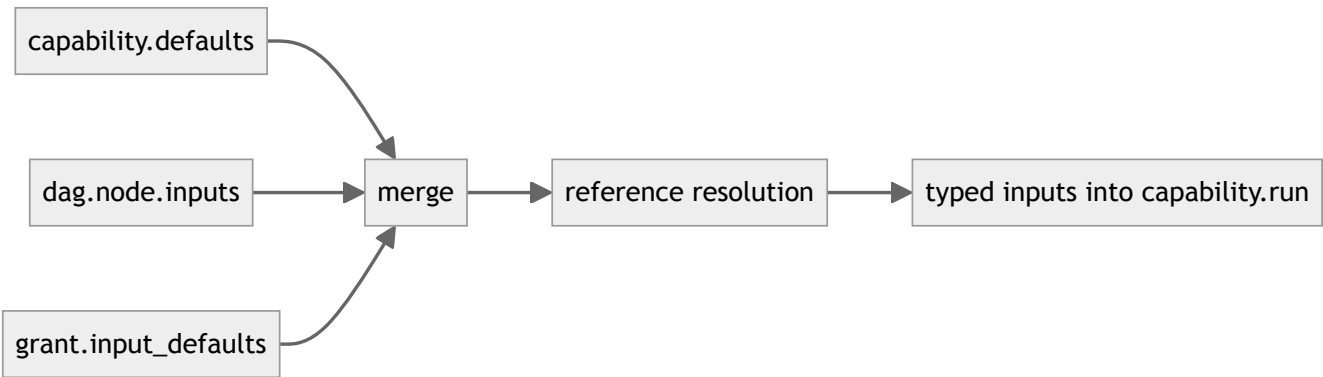
11. Run event kinds

Kind	Emitted by	Carries
<code>run.started</code>	Executor	dag snapshot
<code>node.started</code>	Executor	node id, capability id
<code>node.input</code>	Executor	resolved input map
<code>node.output</code>	Executor	output map (PII scrubbed)
<code>node.failed</code>	Executor	error type, message
<code>node.screenshot</code>	Browser worker	object_uri, mime
<code>signal.published</code>	Hub	id, kind, screenshot
<code>signal.resolved</code>	Hub	id, who, payload
<code>run.succeeded</code>	Executor	summary
<code>run.failed</code>	Executor	failed node id
<code>run.cancelled</code>	Executor	who, reason

The frontend subscribes via SSE on `/runs/{id}/events` .

12. Input defaults merge

A capability declares default inputs. A DAG node may override any of them. Reference resolution happens after merge.



Order of precedence: `dag.node.inputs` (non-empty) > `grant.input_defaults` > `capability.defaults` . References are resolved last so they can target merged values.

13. Capability semantic index

`CapabilityIndex` embeds each granted capability's description + tags via `sentence-transformers` (`BAAI/bge-small-en-v1.5`) and writes per-tenant FAISS shards to disk. It is refreshed when an admin adds, updates, or removes a grant.

Status note: v1 maintains the index on every grant change but the planner does *not* consult it at plan-time today — the prompt builder still renders every granted capability for the model. Wiring `CapabilityIndex.search(prompt, k=12)` into `PromptBuilder` is a planned change (Phase 1, listed in the roadmap) that activates the read path.

14. Test harness

Tests live alongside the source under `aakar/tests/` . Categories:

- **Unit** — pure functions in the planner, validator, prompt rendering, discovery scoring.
- **Capability** — each capability has its own test file driving a fake browser or HTTP backend.
- **API integration** — FastAPI TestClient, real database, faked LLM and browser.
- **End-to-end** — small set of scripted runs against a local mock site.

Integration tests must hit a real database, not mocks. Mock and prod divergence has masked broken migrations in the past.

15. Reading guide

- For intent and boundaries, read the HLD.
- For request and run lifecycles plus the schema, read the backend doc.
- For UI shape and state, read the frontend doc.
- For what comes next, read the roadmap.